

Product Requirement Document (PRD)

Project Name: AbstractiFy

Tagline: *De-jargonize research. Find consensus. Map the science.*

Version: 1.0.0

Status: Approved for Core Development

Author: AI Lead & System Architect

1. Executive Summary

1.1 Context & Evolution

The current prototype of our **Research Paper Intelligence System** runs successfully as a localized Jupyter Notebook. It leverages a static DataFrame of research abstracts, normalizes their embeddings, creates a flat FAISS index (faiss.IndexFlatIP), and runs comparative prompts using standard API patterns.

While functional, it is structurally constrained to local, pre-loaded datasets and relies on simple prompt generation. **AbstractiFy** is the evolution of this prototype into an end-to-end, free-tier research portal. It introduces global API integrations, automated structural metadata extraction, network citation graphing, and structured multi-paper meta-analysis.

1.2 Core Mission

To democratize state-of-the-art academic research intelligence. By utilizing 100% free-tier, open-access scholarly databases and lightweight, local, or free developer API-driven Large Language Models (LLMs), AbstractiFy offers individual researchers, graduate students, and independent developers the processing power of premium institutional software (such as Elicit, Consensus, and Scite) at zero cost.

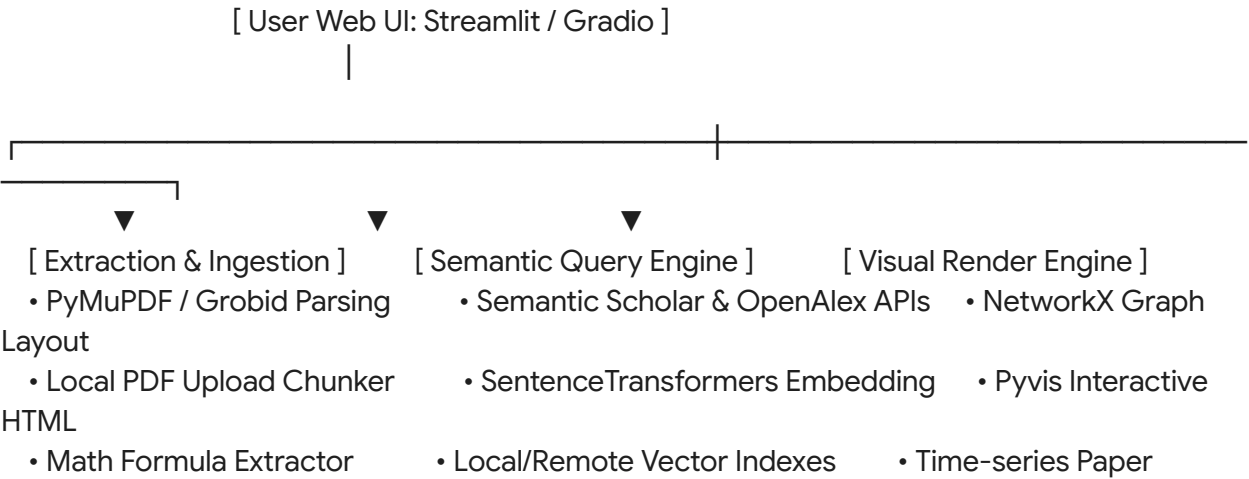
1.3 Key Value Prop

- **Zero Cost:** No subscription models. Every underlying component relies on open-source packages and free academic APIs.
- **Consensus-Driven:** Rather than presenting isolated documents, the tool aggregates the collective scientific stance on query assertions.
- **Visually Explorable:** Maps the historical lineage of papers to show how methodologies mutate over time.

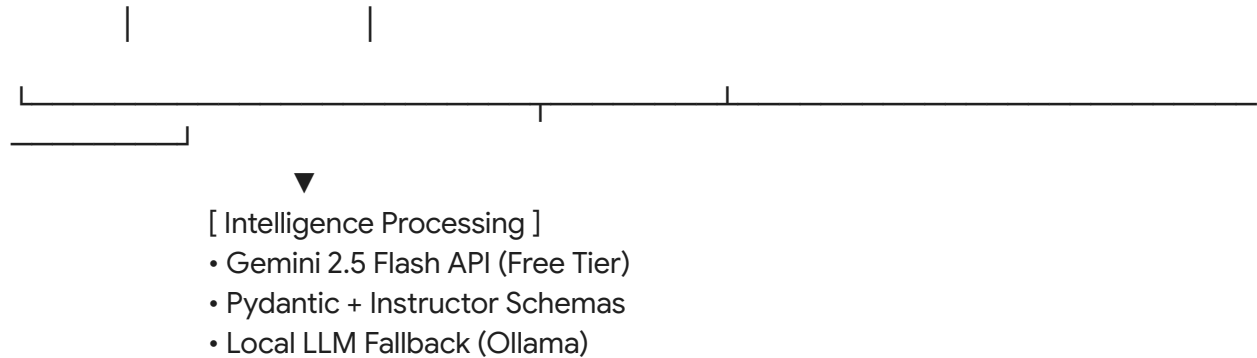
2. Target Audience & User Personas

Persona	Role	Primary Pain Point	Core AbstractiFy Solution
Graduate Student (e.g., Jane)	PhD Candidate, Biology	Spends 15+ hours compiling literature reviews; struggles to find out if methodologies have been debunked.	Consensus Meter to identify scientific consensus and Smart Citations to filter out contested methodologies.
Independent R&D Engineer (e.g., Alex)	Machine Learning Specialist	Needs to extract precise hyperparameter ranges, metrics, and dataset sizes across 20 competing architectures.	Study Comparison Matrix parsing high-dimensional vector representations into structured tables automatically.
Undergraduate Student (e.g., Liam)	Computer Science Major	Dense math equations ($W_{ij} =$) and terminology make papers highly intimidating to read.	In-Document Interactive Chat with a specialized Equation De-jargonizer for inline formulas.

3. Tech Stack & Integration Architecture



Lineage



3.1 APIs & Data Aggregators (Free Tier)

1. **Semantic Scholar Academic Graph API:** Used to retrieve metadata, paper abstracts, citation networks, and citation intents dynamically across 200M+ papers.
2. **OpenAlex API:** A completely open, multi-entity bibliographic catalog used as a high-throughput fallback for comprehensive citation mapping.

3.2 Processing, Vectorization & ML Core

1. **Embedding Generation:** HuggingFace sentence-transformers/all-MiniLM-L6-v2 (384-dimensional dense vectors, lightweight enough to run locally on CPU/GPU).
2. **Local Vector Search:** Facebook AI Similarity Search (FAISS - CPU build) for lightning-fast localized indexing of loaded paper sub-sections.
3. **Structured Extraction LLM:** gemini-2.5-flash-preview-09-2025 (using the free Google Developer API key) for structured JSON generation, translation, and summary synthesis.
4. **Local Fallback Option:** Ollama running Llama-3 or Mistral offline for users demanding absolute data privacy.

3.3 Visual and UI Layout Libraries

1. **Frontend App:** Streamlit or Gradio for rapid development of reactive layouts.
2. **Graph Generation:** NetworkX for calculating node centrality and relationships within citation graphs.
3. **Interactive Visual Rendering:** Pyvis to compile graphs directly into clean, interactive, draggable HTML/JS network representations.

4. Detailed Feature Specifications

4.1 Hybrid Semantic Search & Global Ingestion

- **Description:** The search engine must look up concepts globally instead of reading from a pre-made local index. It combines sparse and dense search patterns to return accurate results for mathematical expressions and specialized terms.
- **Specification Requirements:**

- Query the Semantic Scholar search API with keyword structures.
- Extract top K ($K = 10$ by default) matching documents with complete metadata (Title, Authors, Year, Abstract, DOI, Citation Counts).
- Run local L2-normalized vector similarity (faiss.normalize_L2 and faiss.IndexFlatIP) against the retrieved collection to re-rank documents based on semantic distance to the user's thesis question.

4.2 Study Comparison Matrix (Elicit Style)

- **Description:** Auto-extract design parameters across all retrieved papers to construct a structural matrix.
- **Functional Requirements:**
 - Interface with the LLM using a structured schema parser (such as the instructor library paired with Pydantic validation).
 - For each document, the system must parse the Abstract and target sections, returning a strict JSON object matches this schema:
class StudyDetails(BaseModel):
 dataset_or_sample_size: str # e.g., "N=1,024 chest X-rays"
 core_methodology: str # e.g., "Residual CNN with attention gating"
 primary_outcomes: str # e.g., "94.2% sensitivity, +2.1% AUC over baseline"
 limitations: str # e.g., "Susceptible to out-of-distribution noise"
 - Compile the individual JSON responses into a unified, interactive Pandas DataFrame and display it inside the Streamlit frontend as a spreadsheet matrix with CSV export capability.

4.3 The Consensus Meter (Consensus Style)

- **Description:** Aggregates findings from multiple papers to answer binary and categorical scientific hypotheses (e.g., "*Does physical exercise decrease beta-amyloid accumulation?*").
- **Functional Requirements:**
 - Identify if the user's query is structural or assertion-based.
 - Retrieve the top K relative papers and segment their abstracts.
 - Execute a parallel batch classification prompt requesting the model to classify each paper as:
 1. **YES / Supports** (the paper's results confirm the assertion).
 2. **NO / Contradicts** (the paper's results reject the assertion).
 3. **Mixed / Neutral** (no definitive stance or inconclusive data).
 - Compute the global percentage:

$$\text{Consensus Score} = \frac{N_{\text{Supports}}}{N_{\text{Supports}} + N_{\text{Contradicts}}} \times 100\%$$

- Render a progress-bar visualizer showing the balance of consensus along with a short, 150-word synthesis abstract written by the LLM summarizing the overall scientific landscape.

4.4 In-Document Interactive Chat & Equation Explainer (SciSpace Style)

- **Description:** Allows researchers to upload local PDFs, split them into structural sections (not just arbitrary text chunks), and query mathematical formulations, code fragments, or figures directly.
- **Functional Requirements:**
 - **PDF Parser:** Use PyMuPDF to parse structural coordinates. Filter multi-column layouts into unified reading-order strings.
 - **Local FAISS Indexer:** Automatically parse structural elements like tables and section headers, assigning metadata tags before vectorizing and adding them to a local in-memory FAISS index.
 - **Equation Explainer:** Implement a regex-based parser that identifies LaTeX equations enclosed in \$ or \$\$ symbols. When selected by the user, the app runs a prompt: *"Break down this formula: $X = \dots$ explain every single variable, state what the overall equation solves, and explain it with a simple physical analogy."*

4.5 Smart Citation Context Analysis (Scite Style)

- **Description:** Evaluates citation counts semantically to distinguish between background mentions, methodological adoption, or critical contradictions.
- **Functional Requirements:**
 - Query Semantic Scholar's specialized citationIntents API endpoint.
 - Extract the contextual snippet containing the target paper's ID.
 - Group citation metrics into three tabs on the UI:
 - **Supporting Snippets (N):** Instances where citing papers validated or reproduced results.
 - **Contradicting Snippets (N):** Instances where citing papers pointed out inaccuracies or failed to reproduce findings.
 - **Mentioning Snippets (N):** Standard baseline references in background literature.

4.6 Interactive Network Graphs & Lineage (ResearchRabbit Style)

- **Description:** Displays an interactive web of citation nodes, charting chronologically how papers build upon one another.
- **Functional Requirements:**
 - Build a Graph structure ($G = nx.DiGraph()$) using NetworkX.
 - Add the "Seed Paper" as the core central node.
 - Query OpenAlex/Semantic Scholar for referencing and referenced-by nodes.
 - Limit the depth to $D = 2$ to prevent UI bottlenecking, prioritizing nodes with the highest citation weight.
 - Map node colors to publication years (using a sequential color scheme like *viridis*), and map node size proportionally to citation counts.
 - Output the graph using Pyvis, enabling users to click a node to instantly load its abstract and study matrix parameters into the main app dashboard.

5. User Interface & Interactive Layout Design

```
+-----+
| [AbstractiFy Logo]  [ Search Query: "SNN vs CNN on Edge Devices" ] |
+-----+
|                                     |
| LEFT PANEL: Search Results  | CENTER PANEL: Document Workspace |
|                                     |
| 1. Paper A (94% Match)      | +-----+ | | |
| - Consensus: Supports      | | [Consensus Meter: 85% Yes] | |
| 2. Paper B (89% Match)      | +-----+ |
| - Consensus: Mixed         | |               | |
| 3. Paper C (72% Match)      | | Study Comparison Matrix Table | |
| - Consensus: Contradicts    | | [Export to CSV]           | |
|                               | +-----+ |
|                               | |               | |
|                               | | Interactive Citation Graph | |
|                               | | (Pyvis Draggable Network)  | |
|                               | |               | |
|                               | +-----+ |
+-----+
| RIGHT PANEL: Interactive Reading Assistant |
|                                     |
| - Abstract Explainer chatbot.          |
| - [Input text or select formula]        |
| - Math Breakdown: "$W_{ij}$ represents the synaptic weight..." |
+-----+
```

6. Non-Functional Requirements (NFRs)

- **Performance & Latency:**
 - Dynamic global search queries through OpenAlex and Semantic Scholar must resolve in under **3.0 seconds** on standard broadband networks.
 - Local vector extraction and re-indexing of a 20-page PDF using lightweight MiniLM embeddings must execute in under **5.0 seconds**.
- **API Resiliency (Exponential Backoff):**
 - Implement standard exponential backoff retry algorithms for external API interactions (such as the Gemini API and database connections) to gracefully handle rate limits on the free tiers:

$$\text{Retry Delay} = 2^{\text{retry_count}} \text{ seconds}$$

- If all retries fail, display clear fallback notifications in the UI instead of crashing the system.
- **Data Security & Privacy:**
 - No user uploaded documents are ever indexed permanently into third-party databases. All vectorization indexes (FAISS) remain resident in ephemeral local runtime memory and are cleared when the session closes.

7. Phased Implementation Roadmap

Phase 1: Core Framework Integration (Sprint 1)

- Refactor the prototype notebook into a structured Streamlit app.
- Set up Semantic Scholar and OpenAlex API wrapper modules.
- Integrate SentenceTransformers to handle dynamic multi-document re-ranking over standard FAISS indexes.

Phase 2: Metadata Extraction & Consensus (Sprint 2)

- Write Pydantic parser schemas using the free Gemini API for automated parameter extraction.
- Build the study comparison spreadsheet rendering matrix.
- Integrate the Sentiment Classification engine to generate the global Consensus Meter.

Phase 3: Interactive PDF & Smart Citations (Sprint 3)

- Implement structural PDF chunking using PyMuPDF.
- Implement the custom Equation Explainer interface.

- Incorporate Semantic Scholar's citationIntents API to drive smart peer feedback indicators.

Phase 4: Network Visualization (Sprint 4)

- Hook up NetworkX structural graphs linked dynamically to open APIs.
- Render interactive draggable Pyvis canvas widgets within the UI.
- Run end-to-end regression testing, ensuring no slow-downs or layout breaks across mobile and web interfaces.